



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

SISTEMAS DISTRIBUIDOS

Laboratorio S08

Saga Orquestada y Circuit Breaker

Nombre del alumno:	Yukioayax Canek Gabriel Hernández
Profesor:	Oscar Manuel Ruiz Hurtado
Grupo:	2
Semestre en curso:	2027-1

Ciudad Universitaria, CDMX

Índice

1. Introducción **2**

2. Parte 0: el problema **2**

3. La saga **3**
3.1. Los pasos y su compensación 3
3.2. Diagrama de estados de la transferencia 4
3.3. El código clave 4

4. La compensación funcionando **5**
4.1. Cómo provoqué la falla del abono 5
4.2. Los saldos antes y después 6

5. Cuando la compensación también falla **7**

6. El circuit breaker **9**
6.1. De CERRADO a ABIERTO 9
6.2. De ABIERTO a SEMIABIERTO y a CERRADO 10

7. Lo que la saga no te da: aislamiento **11**

8. Conclusiones **12**

1 Introducción

En este laboratorio la transferencia de Mexi Banco, que en la S07 ya cruzaba servicios por HTTP, se volvió una **saga orquestada**, el `@Transactional` del monolito ya no alcanza cuando el cargo lo hace una réplica de `cuenta` y el abono otra, cada salto se confirma solo, y si el abono falla después del cargo el dinero se queda en el aire, por eso `TransferenciaService` ahora guarda en qué paso va, devuelve el cargo cuando el abono falla (la **compensación**) y todas sus llamadas a `cuenta` pasan por un **circuit breaker** hecho a mano, para no seguir golpeando a un servicio que está caído.

2 Parte 0: el problema

Antes de arreglar el código levanté la rama `03-gateway-discovery-balanceo` y creé dos cuentas de prueba, Ana con 1000 y Beto con 500.

```
#91 [transferencia] resolving provenance for metadata file
#91 DONE 0.0s
[+] up 19/19
 ✔ Image mexi-banco-v06-spei                Built                                                388.1s
 ✔ Image mexi-banco-v06-notificacion        Built                                                388.1s
 ✔ Image mexi-banco-v06-discovery           Built                                                388.1s
 ✔ Image mexi-banco-v06-transferencia       Built                                                388.1s
 ✔ Image mexi-banco-v06-gateway             Built                                                388.1s
 ✔ Image mexi-banco-v06-cuenta              Built                                                388.1s
 ✔ Image mexi-banco-v06-movimiento          Built                                                388.1s
 ✔ Volume mexi-banco-v06_datos              Created                                                0.0s
 ✔ Network mexibanco-v06                    Created                                                0.1s
 ✔ Container mexi-banco-v06-discovery-1     Healthy                                               24.9s
 ✔ Container mexi-banco-v06-db-1            Healthy                                               18.8s
 ✔ Container mexi-banco-v06-gateway-1       Started                                               26.6s
 ✔ Container mexi-banco-v06-transferencia-1 Started                                               25.5s
 ✔ Container mexi-banco-v06-cuenta-3        Started                                               31.5s
 ✔ Container mexi-banco-v06-notificacion-1  Started                                               26.2s
 ✔ Container mexi-banco-v06-cuenta-1        Started                                               27.1s
 ✔ Container mexi-banco-v06-movimiento-1    Started                                               25.8s
 ✔ Container mexi-banco-v06-cuenta-2        Started                                               25.9s
 ✔ Container mexi-banco-v06-spei-1          Started                                               26.3s

jacks@YacGab UCRT64 /d/YacGab/Desktop/Sistemas Distribuidos/mexi-banco-s08 (trabajo-s08)
$ docker compose ps
NAME                          IMAGE                          COMMAND                  SERVICE         CREATED         STATUS                   PORTS
mexi-banco-v06-cuenta-1       mexi-banco-v06-cuenta          "java -jar app.jar"      cuenta          5 minutes ago   Up 4 minutes (healthy)   8080/tcp
mexi-banco-v06-cuenta-2       mexi-banco-v06-cuenta          "java -jar app.jar"      cuenta          5 minutes ago   Up 4 minutes (healthy)   8080/tcp
mexi-banco-v06-cuenta-3       mexi-banco-v06-cuenta          "java -jar app.jar"      cuenta          5 minutes ago   Up 4 minutes (healthy)   8080/tcp
mexi-banco-v06-db-1           postgres:17-alpine             "docker-entrypoint.s…"   db              5 minutes ago   Up 5 minutes (healthy)   5432/tcp
mexi-banco-v06-discovery-1    mexi-banco-v06-discovery       "java -jar app.jar"      discovery       5 minutes ago   Up 5 minutes (healthy)   0.0.0.0:8761->8761/tcp, [::]:8761->8761/tcp
mexi-banco-v06-gateway-1      mexi-banco-v06-gateway         "java -jar app.jar"      gateway         5 minutes ago   Up 4 minutes (healthy)   0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp
mexi-banco-v06-movimiento-1   mexi-banco-v06-movimiento      "java -jar app.jar"      movimiento      5 minutes ago   Up 4 minutes (healthy)   8080/tcp
mexi-banco-v06-notificacion-1 mexi-banco-v06-notificacion    "java -jar app.jar"      notificacion    5 minutes ago   Up 4 minutes (healthy)   8080/tcp
mexi-banco-v06-spei-1         mexi-banco-v06-spei            "java -jar app.jar"      spei            5 minutes ago   Up 4 minutes (healthy)   8080/tcp
mexi-banco-v06-transferencia-1 mexi-banco-v06-transferencia  "java -jar app.jar"      transferencia   5 minutes ago   Up 4 minutes (healthy)   8080/tcp

jacks@YacGab UCRT64 /d/YacGab/Desktop/Sistemas Distribuidos/mexi-banco-s08 (trabajo-s08)
$ curl -s -X POST http://localhost:8080/api/cuentas -H "Content-Type: application/json" -d '{"clabe":"S08111111", "titular":"Ana", "saldoInicial":1000}'
echo ""
{"id":1,"clabe":"S08111111","titular":"Ana","saldo":1000}

jacks@YacGab UCRT64 /d/YacGab/Desktop/Sistemas Distribuidos/mexi-banco-s08 (trabajo-s08)
$ curl -s -X POST http://localhost:8080/api/cuentas -H "Content-Type: application/json" -d '{"clabe":"S08222222", "titular":"Beto", "saldoInicial":500}'
echo ""
{"id":2,"clabe":"S08222222","titular":"Beto","saldo":500}

jacks@YacGab UCRT64 /d/YacGab/Desktop/Sistemas Distribuidos/mexi-banco-s08 (trabajo-s08)
$ curl -s http://localhost:8080/api/cuentas/S08111111
{"id":1,"clabe":"S08111111","titular":"Ana","saldo":1000.00}
jacks@YacGab UCRT64 /d/YacGab/Desktop/Sistemas Distribuidos/mexi-banco-s08 (trabajo-s08)
$ echo ""


jacks@YacGab UCRT64 /d/YacGab/Desktop/Sistemas Distribuidos/mexi-banco-s08 (trabajo-s08)
$ curl -s http://localhost:8080/api/cuentas/S08222222
{"id":2,"clabe":"S08222222","titular":"Beto","saldo":500.00}
jacks@YacGab UCRT64 /d/YacGab/Desktop/Sistemas Distribuidos/mexi-banco-s08 (trabajo-s08)
$ echo ""
```

Figura 1: Creación de cuentas y saldos iniciales.

Luego levanté `transferencia` con una pausa de 20 segundos entre el cargo y el abono (`PAUSA_ENTRE_CARGO_Y_ABONO_MS=20000`), lancé una transferencia y durante esa pausa apagué `cuenta`.

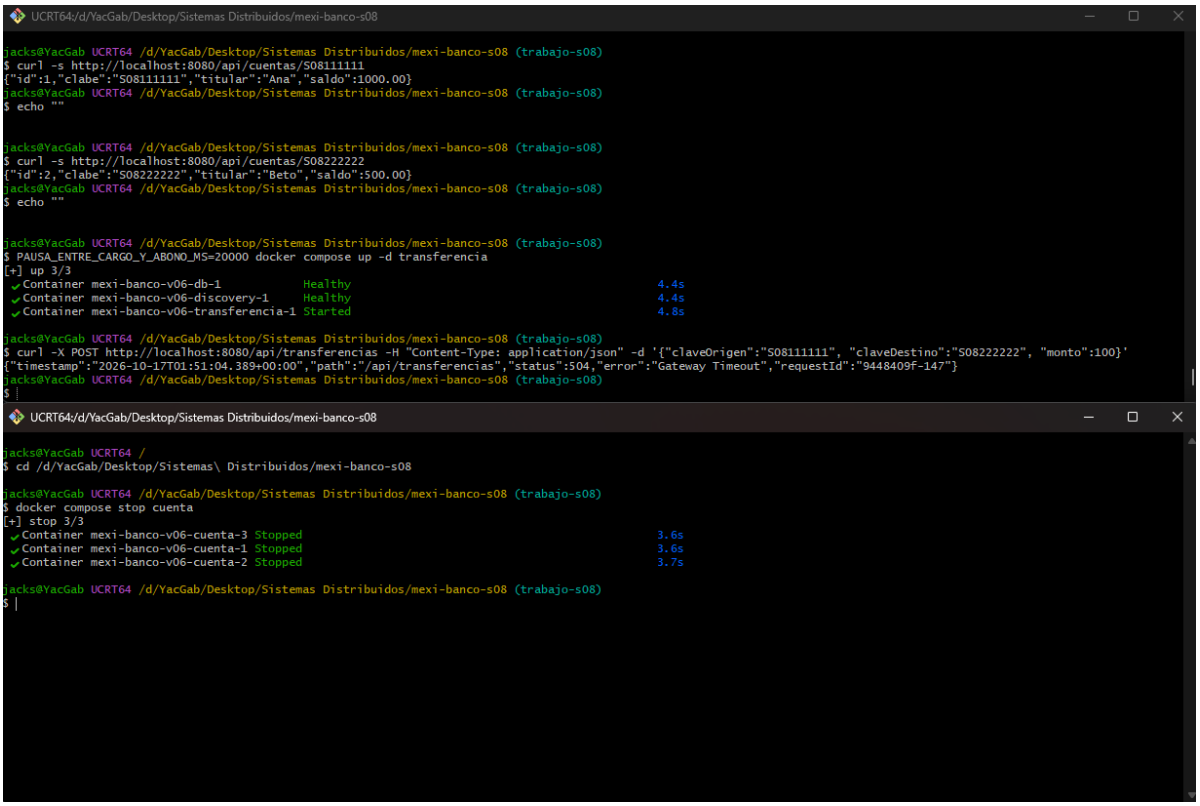


Figura 2: Apagado de `cuenta` durante la pausa de la transferencia.

Al encender otra vez `cuenta` y consultar, el dinero salió de Ana (900) pero nunca llegó a Beto (500), porque el cargo ya se había confirmado en `cuenta` y el abono ya no encontró a nadie, y como cada llamada se confirma sola, no había nada que deshiciera el cargo.

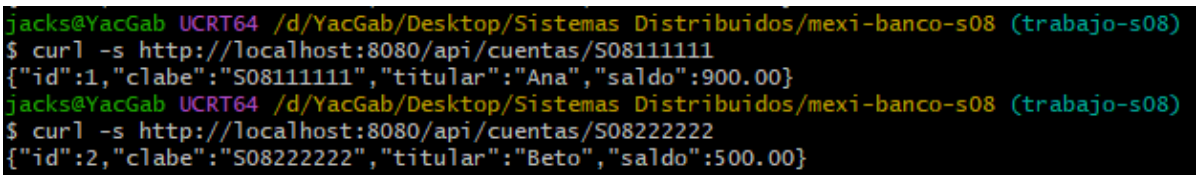


Figura 3: El saldo de Ana bajó y el de Beto no subió: el dinero se quedó en el aire.

3 La saga

3.1 Los pasos y su compensación

`TransferenciaService` es el orquestador, él decide qué paso sigue y qué hacer si uno falla, y cada vez que cambia de paso guarda el estado en la tabla `transferencia`, así, si el servicio se cae a la mitad, la base dice exactamente dónde se quedó.

Paso	Qué hace	Si falla
1. Cargo	consulta que exista el destino y le carga el monto al origen	no se movió dinero, no hay nada que compensar: `FALLIDA` (503)
2. Abono	le abona el monto al destino	se compensa, se le devuelve el monto al origen: `COMPENSADA` (502), y si la devolución también falla, `COMPENSACION_PENDIENTE` (502)
3. Asientos y avisos	registra los dos movimientos y avisa	el dinero ya está donde debe, no se deshace, se queda en `ABONADA` para que se vea que faltan los asientos

3.2 Diagrama de estados de la transferencia

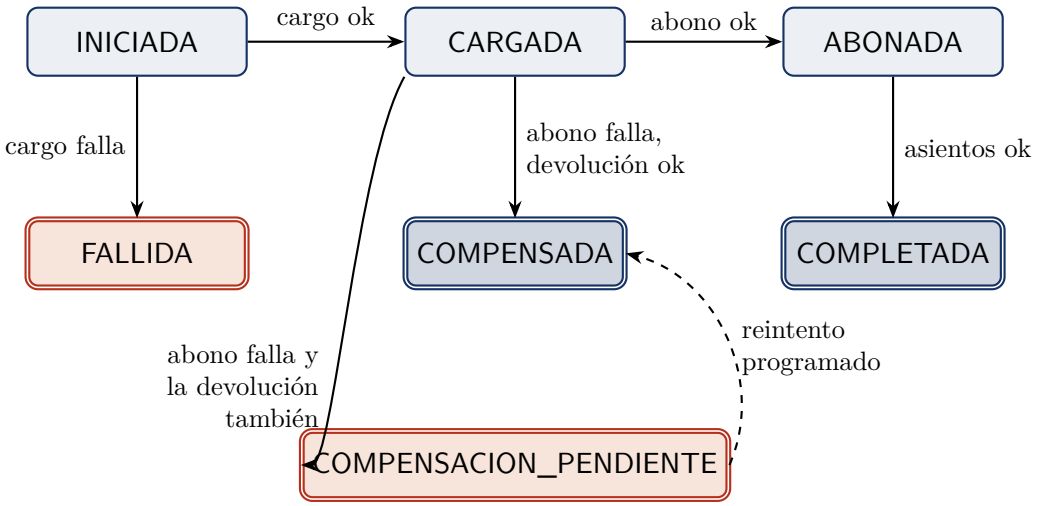


Figura 4: Estados de la transferencia. Los de doble borde son finales; la flecha punteada es el reintento que haría falta para terminar las pendientes.

3.3 El código clave

Lo que cambié está en `transferencia/`: `EstadoTransferencia`, `Transferencia` (con su `estado` y un `detalle` que no se guarda, solo va en la respuesta), `TransferenciaService`, `MiCircuitBreaker`, `TransferenciaController`, `TransferenciaRepository` y el `application.yml`, y en el `docker-compose.yml` solo agregué tres variables a `transferencia`. Cada cambio de estado pasa por un solo método que guarda y deja una línea en el log:

```
private Transferencia cambiarEstado(Transferencia t, EstadoTransferencia nuevo) {
    t.setEstado(nuevo);
    Transferencia guardada = transferencias.save(t);
```

```
    log.info("Transferencia␣{}:␣{}", guardada.getId(), nuevo);
    return guardada;
}
```

El abono va en su propio `try`, porque es el único paso que, si falla, deja algo que deshacer, y antes de llamar a `cuenta` revisa la falla provocada:

```
try {
    if (!fallarAbonoAClabe.isEmpty() && fallarAbonoAClabe.equals(claveDestino)) {
        throw new IllegalStateException("Falla␣provocada␣del␣abono␣(FALLAR_ABONO_A_CLABE
            =...)");
    }
    destino = cbCuenta.ejecutar("abono", () -> cuentas.abonar(claveDestino, monto));
} catch (RuntimeException e) {
    return compensar(t, origen, e.getMessage());
}
```

La compensación le abona el monto al origen y deja los dos asientos, el del cargo y el de la devolución, para que el estado de cuenta explique por qué el saldo bajó y volvió a subir, si la devolución también falla, no se queda en `CARGADA`, sino en `COMPENSACION_PENDIENTE`:

```
try {
    CuentaRemota devuelta = cbCuenta.ejecutar("compensacion",
        () -> cuentas.abonar(t.getClaveOrigen(), t.getMonto()));
    movimientos.registrar(origen, TRANSFERENCIA_ENVIADA, -monto, ..., "a␣...␣(
        transferencia␣N)");
    movimientos.registrar(origen, TRANSFERENCIA_RECIBIDA, monto, ..., "devolucion␣de␣la␣
        transferencia␣N");
    return terminar(t, COMPENSADA, "El␣abono␣fallo␣(...);␣el␣cargo␣se␣devolvio␣a␣la␣
        cuenta␣de␣origen");
} catch (RuntimeException e) {
    return terminar(t, COMPENSACION_PENDIENTE, "...␣la␣devolucion␣queda␣pendiente␣de␣
        reintento");
}
```

Y el controlador ya no contesta 201 a todo, el código HTTP dice cómo terminó la saga y el cuerpo trae el `estado` y el `detalle`: **201** si quedó `COMPLETADA`, **502** si el abono falló (`COMPENSADA` o `COMPENSACION_PENDIENTE`) y **503** si quedó `FALLIDA` porque `cuenta` no estaba disponible. También agregué `GET /transferencias?estado=...` y `GET /transferencias/circuito` para revisar la saga y el circuito desde afuera.

4 La compensación funcionando

4.1 Cómo provoqué la falla del abono

En mi primera versión la falla estaba metida en el código con una CLABE fija, ahora es una variable de entorno, `FALLAR_ABONO_A_CLABE`, si trae una CLABE, todo abono hacia esa cuenta truena a propósito antes de llamar a `cuenta`, y vacía no hace nada, así se prueba la compensación con `cuenta` arriba y sin tocar el código. La pasé al servicio en el `docker-compose.yml` y en el `application.yml`:

```
# docker-compose.yml, servicio transferencia
FALLAR_ABONO_A_CLABE: ${FALLAR_ABONO_A_CLABE:-}
# application.yml
mexibanco.demo.fallar-abono-a-clabe: ${FALLAR_ABONO_A_CLABE:}
```

Para la prueba usé dos cuentas nuevas, Ana `002180000000008001` con 1000 y Beto `012180000000008001` con 500, y levanté `transferencia` con `FALLAR_ABONO_A_CLABE=012180000000008001`, o sea, la cuenta de Beto. Dentro del contenedor la variable está y al arrancar el servicio lo avisa en el log.

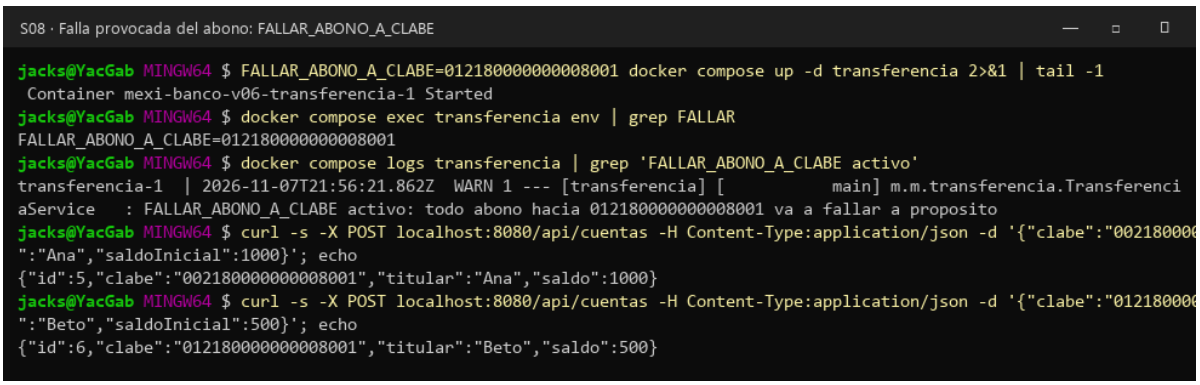


Figura 5: `transferencia` levantado con `FALLAR_ABONO_A_CLABE` apuntando a Beto, y las cuentas de prueba.

4.2 Los saldos antes y después

Mi predicción era que el cargo sí se iba a hacer, Ana iba a bajar a 900 por un momento, el abono a Beto iba a fallar y la saga le iba a regresar los 100, así que al final los dos tenían que quedar igual que al principio. Y eso pasó, antes Ana tenía 1000 y Beto 500, la transferencia de 100 respondió **502** con estado `COMPENSADA` y el detalle de qué falló, y después Ana seguía en 1000 y Beto en 500, en la base, la transferencia 33 quedó con estado `COMPENSADA`.

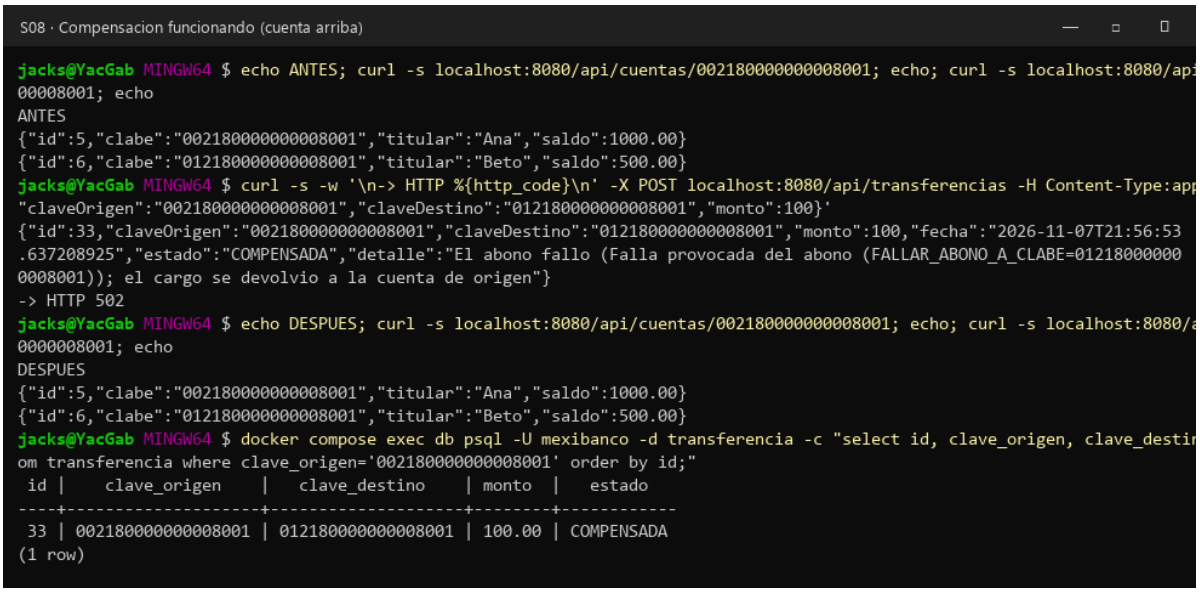


Figura 6: Saldos antes y después de la transferencia de 100 con el abono fallado a propósito, la respuesta 502 y el estado `COMPENSADA` en la base `transferencia`.

En el log se ve cada paso con su hora, `INICIADA`, `CARGADA`, el fallo del abono, la devolución con el saldo de 1000 y `COMPENSADA`, y en el estado de cuenta de Ana quedaron los dos asientos, el cargo de -100 con saldo resultante de 900 y la devolución de $+100$ con 1000.

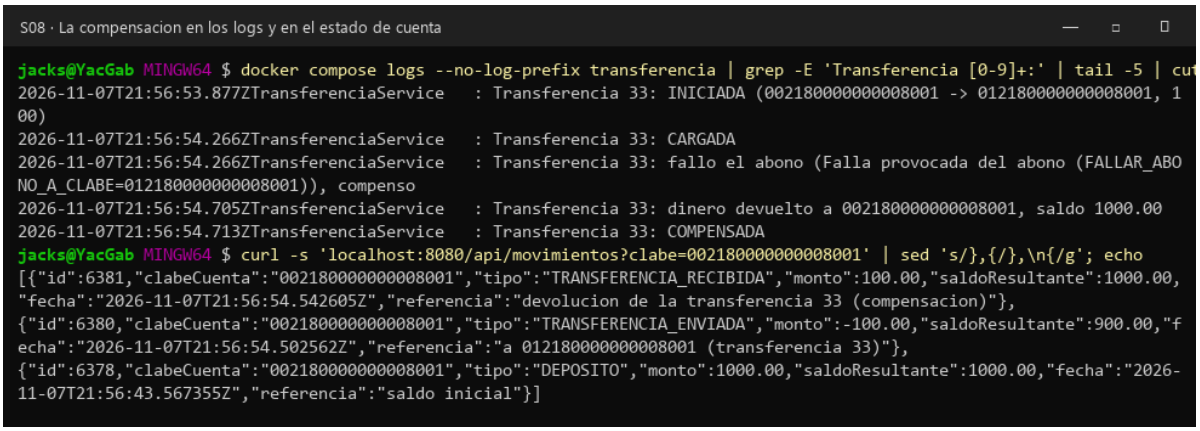


Figura 7: Logs de la saga para la transferencia 33 y estado de cuenta de Ana con el cargo y la devolución.

5 Cuando la compensación también falla

Repetí la Parte 0 con mi versión, `transferencia` con una pausa de 20 segundos, lancé una transferencia de 100 y en la pausa apagué las tres réplicas de `cuenta`. Mi predicción era que el abono iba a fallar, la saga iba a intentar devolver el dinero, y como `cuenta` seguía apagada la devolución también iba a fallar, quedando en `COMPENSACION_PENDIENTE`.

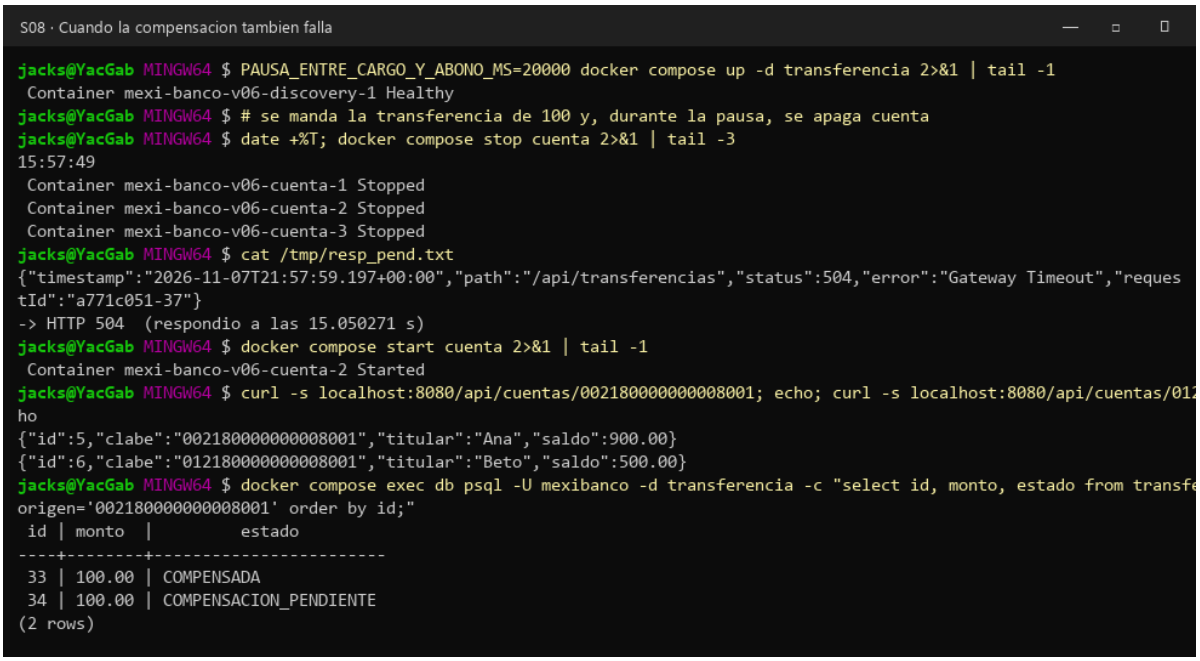


Figura 8: Transferencia de 100 con `cuenta` apagada durante la pausa: el gateway corta a los 15 s con 504, Ana queda en 900 y la transferencia 34 en `COMPENSACION_PENDIENTE`.

Pasó eso, pero con un detalle que no esperaba, el cliente no vio el 502 de mi controlador sino un **504 Gateway Timeout** a los 15 segundos, porque el gateway tiene un tiempo máximo de respuesta y la pausa era de 20, o sea que el gateway se rindió antes de que la saga terminara, aun así, la saga siguió adentro, y la base dice `COMPENSACION_PENDIENTE` para la transferencia 34, Ana quedó en 900 y Beto en 500, los 100 están en el aire pero ahora la base lo sabe.

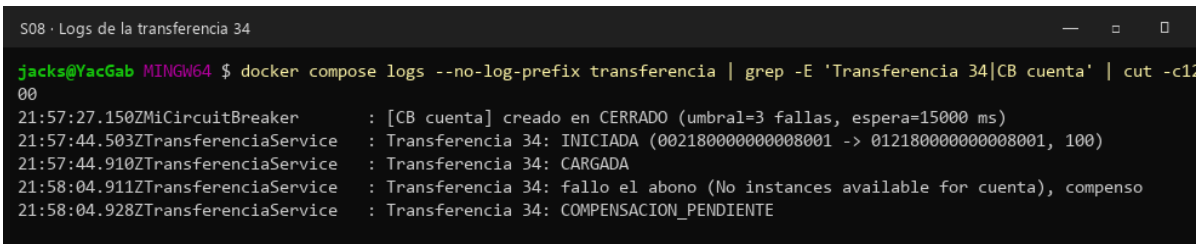


Figura 9: Logs de la transferencia 34: el abono falla con `No instances available for cuenta` y la devolución tampoco pasa.

Cómo la terminaría. Con un reintento programado, un `@Scheduled` que cada cierto tiempo busque las transferencias en `COMPENSACION_PENDIENTE` (para eso ya está `findByEstado`) y vuelva a intentar la devolución, con espera creciente entre intentos (10 s, 20 s, 40 s. . .) para no golpear a `cuenta` justo cuando se está levantando, y cuando por fin pase, que las deje en `COMPENSADA`, y si después de muchos intentos sigue sin pasar, que las marque para que una persona las revise.

Por qué la devolución tiene que ser idempotente. Porque el reintento no sabe si el intento anterior falló antes o después de llegar a `cuenta`, puede que el abono de la devolución

sí se aplicó y lo que se perdió fue la respuesta (justo lo que pasó con el timeout del gateway), si el reintento vuelve a abonar sin más, Ana recibe los 100 dos veces, por eso la devolución tendría que llevar una llave, por ejemplo `compensacion-34`, y `cuenta` tendría que reconocerla y no aplicarla dos veces, igual que el `Idempotency-Key` del SPEI, reintentar sin idempotencia cobra (o en este caso devuelve) dos veces.

6 El circuit breaker

`MiCircuitBreaker` está hecho a mano, todas las llamadas de `transferencia` a `cuenta` (consultar, cargar, abonar y devolver) pasan por su método `ejecutar`, y tiene tres estados:

- **CERRADO**: las llamadas pasan, cada falla seguida suma uno y al llegar a 3 se abre.
- **ABIERTO**: ya no llama a `cuenta`, rechaza al instante con `CircuitoAbiertoException`, y cuando pasan 15 segundos desde que se abrió, la siguiente llamada lo pone en semiabierto.
- **SEMIABIERTO**: deja pasar *una* llamada de prueba, si sale bien se cierra y si falla se vuelve a abrir y empieza otra espera.

Solo cuentan como falla las que dicen que `cuenta` no está sana, que no contestó, que el directorio no tiene instancias o que contestó un 5xx, un 404 o un 422 (la cuenta no existe, saldo insuficiente) es una respuesta normal y no abre el circuito. El umbral y la espera se cambian con `CB_UMBRAL` y `CB_ESPERA_MS`.

```
if (estado == Estado.ABIERTO) {
    long transcurrido = System.currentTimeMillis() - abiertoDesde;
    if (transcurrido < esperaMs) {
        throw new CircuitoAbiertoException("Circuito␣ABIERTO␣hacia␣cuenta:␣...");
    }
    cambiarA(Estado.SEMIABIERTO, "pasaron␣" + transcurrido + "␣ms␣desde␣que␣se␣abrio");
}
if (estado == Estado.SEMIABIERTO) {
    if (pruebaEnCurso) throw new CircuitoAbiertoException("...␣ya␣hay␣una␣prueba␣en␣
        curso");
    pruebaEnCurso = true;
}
```

6.1 De CERRADO a ABIERTO

Con el circuito `CERRADO` apagué `cuenta` y mandé seis transferencias de 10 seguidas. Mi predicción era que las tres primeras iban a tardar porque sí intentan llamar a `cuenta` y esperan el tiempo de conexión, y que de la cuarta en adelante iban a responder de inmediato. Así fue, las tres primeras tardaron unos **2 segundos** cada una (el *connect timeout* de 2 s del cliente HTTP) y respondieron 503, en el log se ve `falla 1/3`, `2/3`, `3/3` y el cambio `CERRADO ->ABIERTO`, y las otras tres respondieron en **0.04 segundos**, porque el circuito ya ni siquiera intentó llamar a `cuenta`, el log dice `rechazo 'consultar destino' sin llamar a cuenta` y cuánto falta para probar.

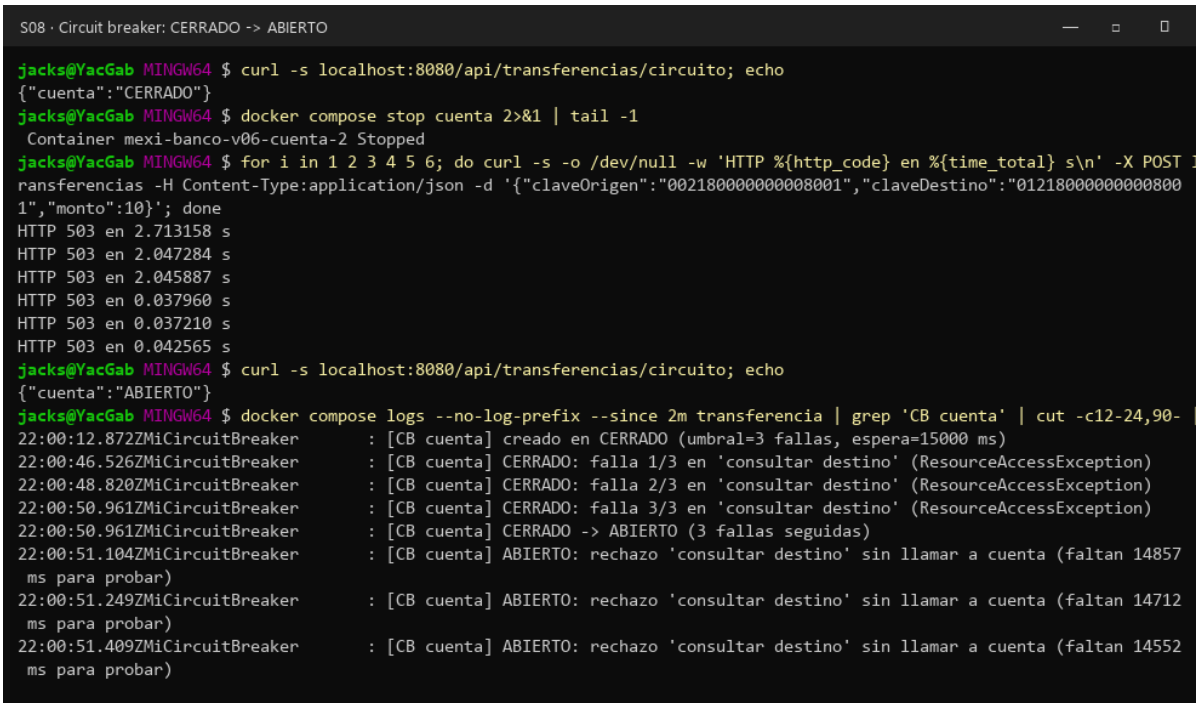


Figura 10: `cuenta` apagada: tres fallas de 2 s, el circuito se abre y las siguientes se rechazan en 0.04 s.

6.2 De ABIERTO a SEMIABIERTO y a CERRADO

Dejé pasar los 15 segundos con `cuenta` todavía apagada y mandé otra transferencia, el circuito pasó a `SEMIABIERTO`, dejó pasar esa llamada como prueba, falló porque el directorio ya no tenía instancias de `cuenta`, y regresó a `ABIERTO`, la transferencia 41 quedó `FALLIDA` con el detalle de que no se movió dinero.

Después encendí `cuenta`, esperé a que estuviera sana y registrada en Eureka, y el circuito seguía `ABIERTO` porque nadie lo había vuelto a probar, la siguiente transferencia lo puso en `SEMIABIERTO`, la llamada de prueba salió bien y pasó a `CERRADO`, esa transferencia (la 42) quedó `COMPLETADA` con 201, y Ana quedó en 890 y Beto en 510 (los 100 de la 34 siguen pendientes, más los 10 de la 42).

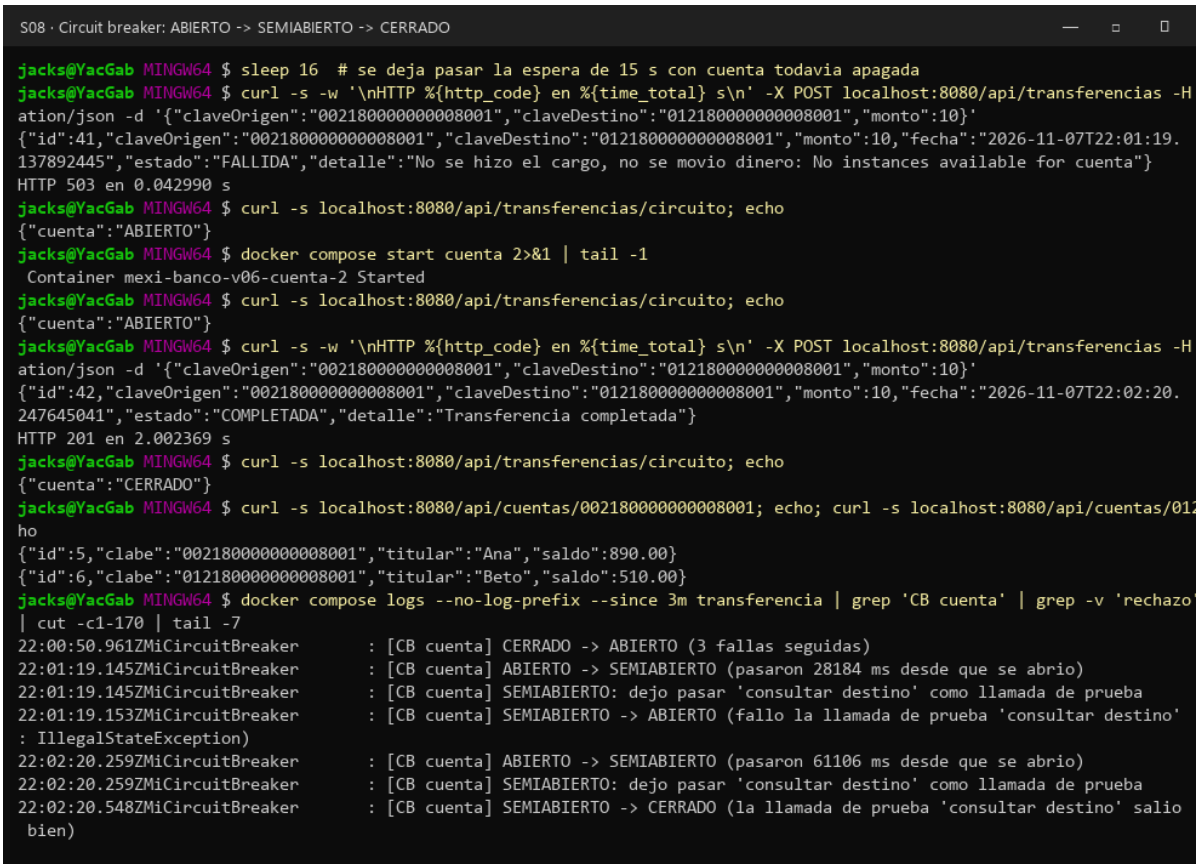


Figura 11: El circuito pasa por los tres estados: ABIERTO, SEMIABIERTO que falla y vuelve a ABIERTO, y con `cuenta` arriba SEMIABIERTO y CERRADO.

7 Lo que la saga no te da: aislamiento

La saga no tiene la "I" de ACID, cada paso se confirma solo y en cuanto se confirma ya lo ve todo el mundo, aunque la saga todavía no termine. Para verlo levanté `transferencia` con la falla provocada hacia Beto y una pausa de 10 segundos, lancé una transferencia de 100 y a los 4 segundos consulté a Ana, antes tenía 890, a la mitad de la saga **790**, y cuando la saga compensó, otra vez 890.

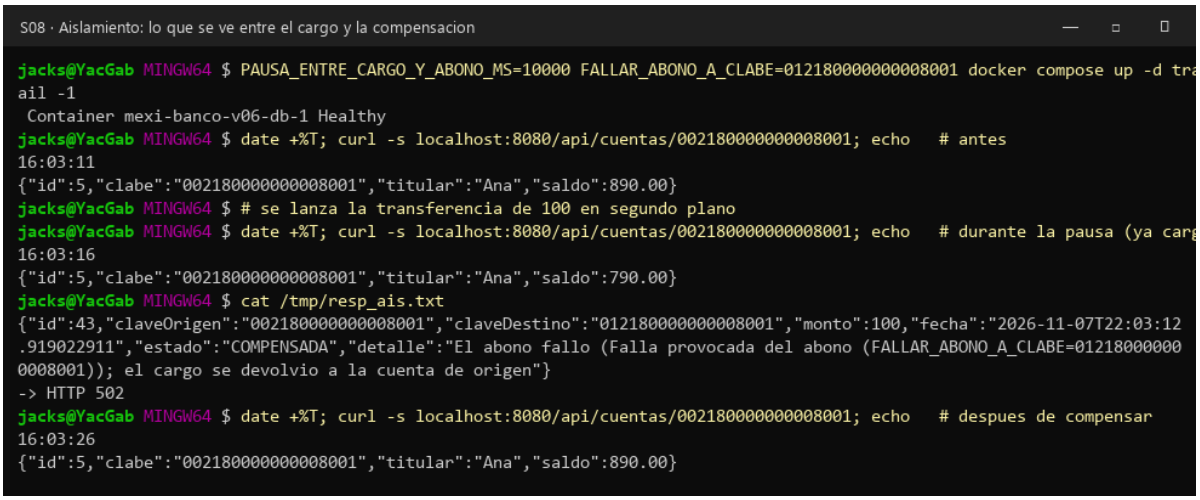


Figura 12: El saldo de Ana antes (890), entre el cargo y la compensación (790) y después (890).

O sea, quien consulta el saldo de origen entre el cargo y la compensación ve un saldo que en realidad nunca debió existir, 100 pesos menos por una transferencia que al final no se hizo, si en ese momento Ana intenta pagar algo con 850 se lo rechazan por saldo insuficiente, o si otro proceso lee ese saldo para un reporte, se lleva un número que luego cambia solo, es una lectura sucia, en el monolito el `@Transactional` lo escondía porque nadie veía el cargo hasta el `COMMIT`, aquí no hay forma de esconderlo, lo más que se puede hacer es ordenar los pasos para que lo que se vea a la mitad sea lo menos dañino, por ejemplo, que el cargo vaya primero y no el abono, porque es peor que alguien vea dinero que todavía no es suyo y lo gaste, o marcar el monto como "retenido" en lugar de descontarlo.

8 Conclusiones

La saga no evita que algo falle a la mitad, lo que hace es que la base sepa en qué paso se quedó y que haya un camino de regreso para cada paso, con la falla provocada vi que el dinero regresa solo y queda escrito en el estado de cuenta, y con `cuenta` apagada vi el caso que sigue abierto, cuando la devolución tampoco pasa la transferencia se queda en `COMPENSACION_PENDIENTE`, y terminarla necesita un reintento que sea idempotente. El circuit breaker, por su parte, hizo que con `cuenta` caída las transferencias dejaran de esperar 2 segundos cada una y respondieran en 0.04, y que cuando `cuenta` regresó, una sola llamada de prueba bastara para cerrarlo. Lo que no se arregla con ninguno de los dos es el aislamiento, entre el cargo y la compensación cualquiera puede ver un saldo que luego se corrige solo.